

1 Derivations, Parse Trees, and Ambiguity

A grammar's four-tuple (V, Σ, P, S) and its production rules specify only what rewriting is *allowed*; they say nothing yet about how S actually turns into a finished string of $L(G)$. This document completes that picture. We define **derivation**, the single rewriting step $\Rightarrow$ and its closure $\Rightarrow^*$ that turns S into w ; draw the **parse tree** that captures the same information with the step order stripped away, and read its **yield** off the leaves; distinguish the two canonical derivation orders, **leftmost** and **rightmost**; and finally examine the grammar $E \rightarrow E + E \mid E * E \mid (E) \mid a$ – broken by design, since nothing in its rules says whether $+$ or $*$ should group first – to show precisely what **ambiguous** means, and what a grammar has to do to avoid it.

1.1 Derivations: From S to a String

A production rule $A \rightarrow \beta$ is a licence, not yet an action – it says you *may* replace one occurrence of A by β inside a longer string of variables and terminals. Use that licence once, and you have taken a **derivation step**; use it over and over, starting from S , and you have a **derivation**.

Derivation, sentential form, and $L(G)$

For $G = (V, \Sigma, P, S)$ and $\gamma_1, \gamma_2 \in (V \cup \Sigma)^*$, write $\gamma_1 \Rightarrow \gamma_2$ (a **derivation step**) if $\gamma_1 = \alpha A \delta$ and $\gamma_2 = \alpha \beta \delta$ for some $A \rightarrow \beta \in P$ and $\alpha, \delta \in (V \cup \Sigma)^*$ – one rule, applied to one variable, somewhere in the string. Write $\Rightarrow^*$ for the reflexive-transitive closure: zero or more steps. Any γ with $S \Rightarrow^* \gamma$ is a **sentential form**, and

$$L(G) = \{w \in \Sigma^* : S \Rightarrow^* w\}$$

– the sentential forms that happen to contain no variables left to rewrite.

Take Lecture 3's matched- a 's-and- b 's grammar, $S \rightarrow aSb \mid \varepsilon$, and derive $aabb$:

$$S \Rightarrow aSb \Rightarrow aaSbb \Rightarrow aa\varepsilon bb = aabb,$$

using $S \rightarrow aSb$ twice and $S \rightarrow \varepsilon$ once, in that order. Every intermediate string – S , aSb , $aaSbb$, $aabb$ itself – is a sentential form; only the last one is also a member of $L(G)$.

A quick historical footnote

This exact rewrite-one-variable-at-a-time notation is what John Backus proposed, and Peter Naur refined, to describe ALGOL 60's syntax in 1960 – **Backus-Naur Form**, still the $A ::= \beta$ notation every language reference manual uses today. It was arrived at completely independently of Chomsky's 1956 linguistic version of the same idea (Lecture 4); two fields invented the identical rewriting rule within four years of each other, for entirely different reasons.

Exercises: Derivations

1. Using Lecture 3's balanced-parentheses grammar, $S \rightarrow (S)S \mid \varepsilon$, write out a full derivation of $((()))()$, naming the rule used at each step.
2. Is $aaabb$ a sentential form of the matched- a 's-and- b 's grammar above? (Hint: every sentential form here has exactly as many a 's to the left of the lone S as b 's to its right – check that invariant before trying to build a derivation.)
3. Write $L(G)$ in set-builder notation for the matched- a 's-and- b 's grammar, and confirm $aabb$ from the worked example actually belongs to it.

1.2 Parse Trees and the Yield of a Tree

A derivation is a list – order written on the page, even where the order carries no real information. The $aabb$ derivation above was in fact forced the whole way: at every sentential form there was exactly one variable left, so there was never a choice of which one to rewrite next. Draw the same rewriting as a picture instead of a list, and that irrelevant order disappears entirely. The picture is the **parse tree**.

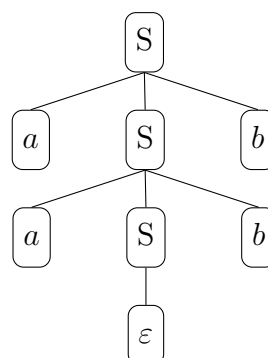
Parse tree and yield

A **parse tree** for a derivation $S \Rightarrow^* w$ under $G = (V, \Sigma, P, S)$ is a tree with:

- root labeled S ;
- every internal node labeled some $A \in V$, whose children – read left to right – are labeled $X_1, \dots, X_k$ for some rule $A \rightarrow X_1X_2 \cdots X_k \in P$ used at that node (a single child labeled ε if the rule used was $A \rightarrow \varepsilon$);
- every leaf labeled a terminal (or ε).

The **yield** of a parse tree is the string obtained by reading its leaf labels left to right, with any ε leaf contributing nothing.

The $aabb$ derivation above builds this tree:



Read the leaves left to right – a, a, ε, b, b – and the yield is $aabb$, exactly the string the derivation produced. That will always hold: whatever order a derivation rewrites variables

in, its parse tree's yield is the derived string, since the tree only remembers *which* rule fired at *which* node, never when.

Exercises: Parse Trees

1. Draw the parse tree for your derivation of $((\))()$ from the previous exercise set, using the balanced-parentheses grammar. Read off its yield and confirm it matches $((\))()$.
2. How many internal nodes does the $aabb$ tree above have, and how does that number relate to the number of a 's (equivalently b 's) in the string? Predict the answer for a^5b^5 before drawing it.
3. Could two *different* derivations of $aabb$ from this same grammar ever produce two *different* parse trees? (Hint: revisit the "forced the whole way" remark that opened this subsection.)

1.3 Leftmost and Rightmost Derivations

The matched- a 's-and- b 's grammar never actually offered a choice of *which* variable to rewrite next, because its sentential forms never held more than one variable at a time. Lecture 3's arithmetic-expression grammar is not so polite:

$$E \rightarrow E + E \mid E * E \mid (E) \mid a.$$

The sentential form $E + E$ already has two variables sitting side by side. Rewrite the left one first, or the right one first – both are legal derivation steps, and a derivation has to pick one at every stage that offers more than one variable.

Leftmost and rightmost derivations

A derivation is **leftmost** if, at every step, the variable rewritten is the leftmost variable in the current sentential form. It is **rightmost** if, at every step, it is always the rightmost variable. (In general a sentential form's variables can be rewritten in *any* order – leftmost and rightmost are simply the two canonical, easiest-to-describe choices.)

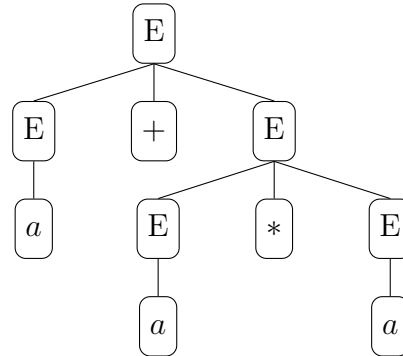
Derive $a + a * a$ two ways, always building the reading "add a to the product $a * a$ " – i.e. $+$ as the outermost operator. **Leftmost** (always the leftmost E):

$$E \Rightarrow E + E \Rightarrow a + E \Rightarrow a + E * E \Rightarrow a + a * E \Rightarrow a + a * a.$$

Rightmost (always the rightmost E), starting from the very same first rule:

$$E \Rightarrow E + E \Rightarrow E + E * E \Rightarrow E + E * a \Rightarrow E + a * a \Rightarrow a + a * a.$$

Both end at the same string, by two genuinely different step orders. Draw either one as a tree and you get the identical picture:



Leftmost derivation is exactly a pre-order, always-expand-leftmost-first walk of building this tree top-down; rightmost is the mirror image, always expanding the rightmost unfinished branch first. Compilers care about this distinction directly: a top-down (LL) parser constructs a leftmost derivation as it reads left to right; a bottom-up (LR) parser constructs a rightmost derivation, but discovers it in reverse, from the leaves up.

Exercises: Leftmost and Rightmost Derivations

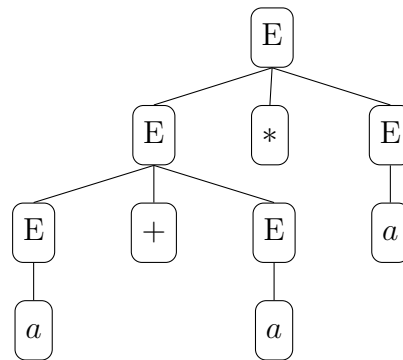
1. Write the leftmost derivation of $(a + a)$ using the arithmetic-expression grammar above.
2. For the tree drawn in this subsection, write down a derivation of $a + a * a$ that is *neither* leftmost nor rightmost – legal, but rewriting variables in some other order. (Hint: at the very first step you have no choice, but by the second step $E + E * E$ you do.)
3. Take the balanced-parentheses grammar $S \rightarrow (S)S \mid \varepsilon$ and your derivation of $(())()$ from Section 1.1's exercises: was it already leftmost, rightmost, both, or neither? Compare against Exercise 3 of that same exercise set.

1.4 Ambiguous and Unambiguous Grammars

Lecture 3 called the arithmetic-expression grammar ambiguous on sight and deferred the proof: “nothing in the rules says whether $+$ or $*$ groups first.” We now have exactly the machinery to make that precise – exhibit *two different* parse trees for the same string, both fully legal under the grammar’s own rules.

Section 1.3 built the tree for “add a to the product $a * a$.” Here is the other reading of $a + a * a$, “multiply $a + a$ by a ” – $*$ as the outermost operator – via its own leftmost derivation:

$$E \Rightarrow E * E \Rightarrow E + E * E \Rightarrow a + E * E \Rightarrow a + a * E \Rightarrow a + a * a,$$



Same grammar, same string $a + a * a$, same yield – but two structurally different trees, reached by two different *leftmost* derivations. (Comparing leftmost derivations specifically is the standard test, precisely because a parse tree and its leftmost derivation determine each other uniquely – unlike derivations in general, where many orders build one tree, as Section 1.3 showed.)

Ambiguous and unambiguous grammars

A context-free grammar G is **ambiguous** if some $w \in L(G)$ has two or more distinct parse trees – equivalently, two or more distinct leftmost derivations of w . G is **unambiguous** if every $w \in L(G)$ has exactly one parse tree.

This is not bookkeeping. The first tree reads as $a + (a * a)$, the second as $(a + a) * a$ – feed real numbers through in place of a and the two readings generally disagree on the value. An ambiguous grammar for an expression language is a compiler bug waiting to happen, the very same dangling-else disease Lecture 1 raised for compiler rules, now caught in a grammar instead.

Fixing it, as Lecture 3 promised, just means encoding precedence directly into the rule shape so no sentential form is ever free to branch two ways:

$$E \rightarrow E + T \mid T, \quad T \rightarrow T * F \mid F, \quad F \rightarrow (E) \mid a.$$

A new layer of variables, T for “term” and F for “factor,” forces every $*$ to bind before any $+$ can see it – under this grammar, $a + a * a$ has exactly one parse tree, the $a + (a * a)$ reading.

Where the undecidability comes from

Telling *one* grammar apart from ambiguous is one thing; telling whether an *arbitrary* given grammar is ambiguous turns out to be undecidable in general – a result usually credited to two papers published independently in the very same month, October 1962: R. W. Floyd’s “On Ambiguity in Phrase Structure Languages” and D. G. Cantor’s “On the Ambiguity Problem of Backus Systems.”

To think!!

Rewriting the arithmetic grammar removed its ambiguity without shrinking $L(G)$ at all – same language, unambiguous grammar. Is that always possible? Revisit Lecture 4's exercise on $L = \{a^i b^j c^k : i = j \text{ or } j = k\}$ with this question in mind: could *every* grammar for some languages be forced to stay ambiguous, no matter how it is rewritten?

Exercises: Ambiguous and Unambiguous Grammars

1. Using the fixed grammar $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow (E) \mid a$, write the leftmost derivation of $a + a * a$ and draw its parse tree. Confirm it matches the $a + (a * a)$ tree from Section 1.3, not the $(a + a) * a$ tree drawn above.
2. Is the balanced-parentheses grammar $S \rightarrow (S)S \mid \varepsilon$ ambiguous? Build the one parse tree you get for $(())()$ and argue informally that no other split of the string is possible under these rules. (Hint: at any point in the string, how many ways can the very next symbol, an opening parenthesis, be matched to its partner?)
3. The original arithmetic grammar also leaves $a + a + a$ with two distinct parse trees, even though $+$ is the only operator involved. Draw both, and say in one sentence what they disagree about that the fixed grammar's $E \rightarrow E + T \mid T$ rule settles.

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (HMU) §5.1.3 “Derivations Using a Grammar” and §5.1.6 “Sentential Forms” – HMU's own version of Section 1.1's $\Rightarrow$ and $\Rightarrow^*$.
2. (HMU) §5.1.4 “Leftmost and Rightmost Derivations” – matches Section 1.3 directly, in the same terms.
3. (HMU) §5.2 “Parse Trees” – the tree and yield definitions of Section 1.2, HMU's way.
4. (HMU) §5.4 “Ambiguity in Grammars and Languages,” specifically §5.4.1 “Ambiguous Grammars,” §5.4.2 “Removing Ambiguity From Grammars,” and §5.4.4 “Inherent Ambiguity” – matches Section 1.4, including the precedence-fixing trick and the open question posed in this document's “To think!!” box.

5. (SIPSER) §2.1 “Context-Free Grammars” – Sipser’s single-section treatment of derivations, leftmost derivation, parse trees, and ambiguity, already flagged in Lecture 3’s notes as picking up exactly where that document stopped.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 Simplification of Context-Free Grammars

Consider the unambiguous arithmetic grammar $E \rightarrow E+T \mid T, T \rightarrow T * F \mid F, F \rightarrow (E) \mid a$, built by encoding operator precedence directly into a hierarchy of variables. Look closely and it is carrying dead weight nobody asked for – $E \rightarrow T$ and $T \rightarrow F$ are productions that do nothing but hand control to another variable, and a grammar assembled by hand, or worse, generated mechanically, routinely accumulates variables that can never fire at all. This document is **simplification**: three surgical passes over a grammar's production rules – strip out **useless symbols** that can never appear in any derivation of a terminal string, strip out **ϵ -productions**, and collapse **unit productions** like this very grammar's $E \rightarrow T$ – each pass provably leaving the language untouched. The three passes look independent. They are not: get their order wrong and one pass quietly undoes the work of another, a trap this document walks straight into before showing the way out.

1.1 Eliminating Useless Symbols

Start with the simplest kind of dead weight: a variable that can sit in a grammar's production list forever without ever contributing a single derivation of an actual string.

Useful, generating, and reachable symbols

For $G = (V, \Sigma, P, S)$, a symbol $X \in V \cup \Sigma$ is:

- **generating** if $X \Rightarrow^* w$ for some $w \in \Sigma^*$ (every terminal is generating, trivially, in zero steps);
- **reachable** if $S \Rightarrow^* \alpha X \beta$ for some $\alpha, \beta \in (V \cup \Sigma)^*$;
- **useful** if there is some derivation $S \Rightarrow^* \alpha X \beta \Rightarrow^* w$ with $w \in \Sigma^*$ – equivalently, X is both generating *and* reachable.

X is **useless** if it is not useful. A production is useless if it mentions a useless symbol.

Order of removal matters: eliminate non-generating symbols first, then eliminate symbols unreachable in what remains – reversing the two can leave useless symbols behind (see below).

Take Hopcroft, Motwani, and Ullman's own opening example: $G = (\{S, A, B\}, \{a, b\}, P, S)$ with

$$P = \{ S \rightarrow AB \mid a, \quad A \rightarrow b \}.$$

Generating, first. a and b generate themselves; $S \rightarrow a$ makes S generating; $A \rightarrow b$ makes A generating. B has no production at all, so it generates nothing – non-generating. Remove B , and with it any production mentioning B – that deletes $S \rightarrow AB$, leaving $S \rightarrow a, A \rightarrow b$.

Reachable, second, on what remains. S is reachable trivially; S 's only remaining production is $S \rightarrow a$, which mentions no variable at all. A is therefore *not* reachable any more. Remove $A \rightarrow b$. Final grammar: $S \rightarrow a$ – exactly $L(G) = \{a\}$, with every symbol both generating and reachable.

Does the order matter here? Yes – watch what reachable-first does

Run the checks in the other order on the *original* grammar $S \rightarrow AB \mid a$, $A \rightarrow b$. **Reachable, first.** S is reachable; $S \rightarrow AB$ makes A and B reachable too – every symbol of the grammar is reachable, nothing is removed. **Generating, second,** on this unchanged grammar: B still has no production, so it is still removed, taking $S \rightarrow AB$ with it – leaving, again, $S \rightarrow a$, $A \rightarrow b$. But this time that is the *final* answer, because the reachability pass has already run and will not run again. $A \rightarrow b$ is left sitting in the grammar, useless and unreachable, exactly the kind of leftover the whole procedure was supposed to remove. In this toy example the leftover happens to be harmless – A never affects $L(G) = \{a\}$ – but the grammar itself is not simplified, and in general such a leftover branch can be arbitrarily large.

Exercises: Useless Symbols

1. (HMU, Exercise 7.1.1) Find a grammar equivalent to

$$S \rightarrow AB \mid CA, \quad A \rightarrow a, \quad B \rightarrow BC \mid AB, \quad C \rightarrow aB \mid b$$

with no useless symbols. (Hint: run the generating check first – at least one of these four variables never derives a terminal string at all.)

2. Simplify $S \rightarrow aA \mid B$, $A \rightarrow b$, $B \rightarrow Bc$, removing all useless symbols. Which check catches B : generating, or reachable?
3. Simplify $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow b$, $C \rightarrow Sc$. Which variable is useless here, and is it caught by the generating check or the reachable check? Compare against Exercise 2 – the two grammars are useless for structurally opposite reasons.

1.2 Eliminating ε -Productions

Useless symbols are dead weight; an ε -production is a different kind of debt – not wrong, just a production whose entire body can vanish, leaving every neighbouring symbol free to ask “was this variable ever really there?” Untangling that question, once per variable, is the whole job of this pass.

ε -production and nullable variable

A production $A \rightarrow \varepsilon$ is an ε -**production**. A variable A is **nullable** if $A \Rightarrow^* \varepsilon$. Nullable variables are found by induction: A is nullable if $A \rightarrow \varepsilon \in P$ (basis), or if $A \rightarrow C_1C_2 \cdots C_k \in P$ with every C_i already known nullable (induction). To build an equivalent grammar with no ε -productions: for every production $A \rightarrow X_1X_2 \cdots X_n$, add *every* version of it obtained by deleting some nonempty subset of the nullable X_i 's (never deleting *all* of them, unless $n = 0$ already) – then drop the original ε -productions outright.

HMU's own example (Example 7.8): $G = (\{S, A, B\}, \{a, b\}, P, S)$ with

$$P = \{S \rightarrow AB, \quad A \rightarrow aAA \mid \varepsilon, \quad B \rightarrow bBB \mid \varepsilon\}.$$

A and B are nullable directly, from their own ε -productions; that makes S nullable too, since $S \rightarrow AB$ has an all-nullable body. **Every** variable is nullable here. For $S \rightarrow AB$: both symbols nullable, so all three ways of keeping at least one give $S \rightarrow AB \mid A \mid B$. For $A \rightarrow aAA$: only the two A 's are nullable, and a is fixed, so all four presence/absence choices are legal – but deleting the second A or the third A gives the identical string aA , so the four choices collapse to three productions: $A \rightarrow aAA \mid aA \mid a$. Symmetrically, $B \rightarrow bBB \mid bB \mid b$. The two original ε -productions themselves contribute nothing to the new grammar. Final:

$$S \rightarrow AB \mid A \mid B, \quad A \rightarrow aAA \mid aA \mid a, \quad B \rightarrow bBB \mid bB \mid b.$$

Exercises: Eliminating ε -Productions

1. Eliminate the ε -productions from $S \rightarrow XYZ$, $X \rightarrow a \mid \varepsilon$, $Y \rightarrow b \mid \varepsilon$, $Z \rightarrow c$. (Hint: two of the three right-hand-side symbols are nullable – how many versions of $S \rightarrow XYZ$ does that force you to list?)
2. Eliminate the ε -productions from $S \rightarrow AB$, $A \rightarrow aA \mid \varepsilon$, $B \rightarrow bB \mid \varepsilon$, and describe $L(G)$ in set-builder notation before and after.
3. For the grammar in Exercise 1, is the resulting language exactly the original $L(G)$, or $L(G) \setminus \{\varepsilon\}$? Check whether ε itself was ever in $L(G)$ to begin with.

1.3 Eliminating Unit Productions

Lecture 11's fix for the arithmetic grammar's ambiguity added exactly two rules that do nothing but relabel one variable as another – $E \rightarrow T$ and $T \rightarrow F$. Both are **unit productions**, and while they were exactly the right tool for encoding precedence there, they complicate other jobs downstream (Chomsky Normal Form among them), so this pass removes them without touching the language.

Unit production and unit pair

A production $A \rightarrow B$, with $A, B \in V$ both variables, is a **unit production**. (A, B) is a **unit pair** if $A \Rightarrow^* B$ using only unit productions (every (A, A) is trivially a unit pair, by zero steps). To build an equivalent grammar with no unit productions: find every unit pair (A, B) , then for each one add to A every *non-unit* production body that B has – and drop all unit productions from the result.

HMU's running example is exactly Lecture 11's family of grammars, one layer richer – add an identifier variable I underneath F :

$$I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1, \quad F \rightarrow I \mid (E), \quad T \rightarrow F \mid T * F, \quad E \rightarrow T \mid E + T.$$

Unit pairs, built up by the induction in the box above: $(E, E), (T, T), (F, F), (I, I)$ from the basis; $E \rightarrow T$ gives (E, T) ; (E, T) with $T \rightarrow F$ gives (E, F) ; (E, F) with $F \rightarrow I$ gives (E, I) ;

similarly (T, F) and (T, I) from T 's own chain; (F, I) from $F \rightarrow I$ directly. Ten pairs in total. Now, for every pair (A, B) , copy in B 's non-unit bodies as new productions headed by A :

$$\begin{aligned} E &\rightarrow E + T \mid T * F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid IO \mid I1, \\ T &\rightarrow T * F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid IO \mid I1, \\ F &\rightarrow (E) \mid a \mid b \mid Ia \mid Ib \mid IO \mid I1, \quad I \rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1. \end{aligned}$$

Not a single unit production survives, and this grammar generates exactly the same identifiers and expressions as before. Notice what did *not* happen: E 's alternatives grew longer, but E never grew a rule like $E \rightarrow E * E$ – the precedence structure Lecture 11 built by hand survives the whole procedure untouched.

Exercises: Unit Productions

1. Eliminate the unit productions from Lecture 11's own fixed grammar, $E \rightarrow E + T \mid T$, $T \rightarrow T * F \mid F$, $F \rightarrow (E) \mid a$ – a strict subset of the worked example above (no I layer). Compare your E 's final alternatives against the worked example's: they should match once every $Ia, Ib, IO, I1$ alternative is dropped.
2. Is $S \rightarrow A$, $A \rightarrow B$, $B \rightarrow A \mid a$ free of unit productions after applying the procedure? Find all unit pairs first – there is a cycle here, and the definition of unit pair in the box above already tells you how to handle it.
3. Eliminate the unit productions from $S \rightarrow AB \mid C$, $C \rightarrow a$, $A \rightarrow a$, $B \rightarrow b$. (Hint: only one production in this grammar is a unit production to begin with.)

1.4 Does the Order Matter?

Section 1.1 already forced one order question on us – generating before reachable, or the grammar comes out with useless symbols still in it. The bigger version of that question: given all three simplifications, is there a safe order to chain them in? HMU states the answer plainly, and it is *not* the order this document just taught them in:

1. Eliminate ε -productions.
2. Eliminate unit productions.
3. Eliminate useless symbols.

Useless symbols, introduced first in Section 1.1 because the concept is easiest to see in isolation, is applied *last* in practice. The reason is mechanical: eliminating ε -productions can *manufacture new unit productions* that were not in the original grammar at all – a production $A \rightarrow BC$ with C nullable produces, among its options, the brand-new unit production $A \rightarrow B$. Run unit-production elimination before that happens, and it never sees this new production; run it after, and it catches it. Eliminating unit productions, by contrast, only ever copies existing non-unit bodies around – it cannot manufacture a fresh ε -production or a fresh useless symbol out of nothing, so it is safe to leave useless-symbol elimination for last, once no further rewriting is left to expose new dead branches.

Here is that failure made concrete. Take $G = (\{S, A, C\}, \{a, b, c\}, P, S)$ with

$$P = \{S \rightarrow AC \mid b, \quad A \rightarrow a, \quad C \rightarrow c \mid \varepsilon\}.$$

Not a single unit production exists in G as written. **Wrong order – unit productions first.** There is nothing to do: no unit productions are present yet, so this pass is a silent no-op. **Then ε -productions.** C is nullable, so $S \rightarrow AC$ yields both $S \rightarrow AC$ and $S \rightarrow A$ – and $S \rightarrow A$ is a unit production that did not exist a moment ago. The unit-production pass has already run and will not run again, so this new production survives into the “final” grammar

$$S \rightarrow AC \mid A \mid b, \quad A \rightarrow a, \quad C \rightarrow c,$$

which still has a unit production sitting in it, in flat contradiction of what the procedure promised to remove. **Correct order – ε -productions first,** on the same G , reaches this identical intermediate grammar $S \rightarrow AC \mid A \mid b, A \rightarrow a, C \rightarrow c$ – but now the unit-production pass still has its turn. It finds the unit pair (S, A) , copies A ’s non-unit body a over to S , and drops $S \rightarrow A$ itself:

$$S \rightarrow AC \mid a \mid b, \quad A \rightarrow a, \quad C \rightarrow c,$$

genuinely free of unit productions, and (useless-symbol check confirms) free of useless symbols too. Both routes describe the same language, $\{a, b, ac\}$ – the bug in the wrong order is not a wrong answer, it is a grammar that silently fails to be as simplified as promised.

To think!!

The pathological example above shows ε -elimination manufacturing a new unit production. Symmetrically: can eliminating unit *productions* ever manufacture a new ε -production, or a new useless symbol, out of a grammar that had none? Look again at exactly what the unit-production construction in Section 1.3 is allowed to copy, and decide whether HMU’s safe order could have put unit-production elimination *before* ε -elimination instead, for the opposite reason.

Exercises: Order of Simplification

1. Run *useless-symbol elimination first*, before anything else, on Section 1.2’s worked example ($S \rightarrow AB, A \rightarrow aAA \mid \varepsilon, B \rightarrow bBB \mid \varepsilon$). Confirm it is a no-op – then explain in one sentence why that is a coincidence of this particular grammar, not a reason to trust the wrong order in general.
2. Take $G = (\{S, X, Y\}, \{x, y\}, P, S)$ with $S \rightarrow XY \mid y, X \rightarrow x, Y \rightarrow y \mid \varepsilon$ – deliberately built to mirror this section’s pathological example. Run unit-production elimination *before* ε -elimination, and check whether a stray unit production survives into your “final” grammar, the same way $S \rightarrow A$ survived above.
3. Apply the full safe pipeline – ε -productions, then unit productions, then useless symbols – to Exercise 1 of Section 1.3’s exercise set. Confirm the result has none of the three features left, in any order you check them in afterward.

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (HMU) §7.1.1 “Eliminating Useless Symbols” – Example 7.1 and the follow-up remark on reachable-first ordering, matching Section 1.1 exactly, including this document’s worked example and its wrong-order warning.
2. (HMU) §7.1.2 “Computing the Generating and Reachable Symbols” – the inductive algorithms behind the generating/reachable tests used in Section 1.1
3. (HMU) §7.1.3 “Eliminating ε -Productions” – Example 7.8, reproduced in full in Section 1.2
4. (HMU) §7.1.4 “Eliminating Unit Productions” – Examples 7.10 and 7.12 (the I, F, T, E grammar), matching Section 1.3
5. (HMU) §7.1, the paragraph and Theorem 7.14 immediately preceding §7.1.5 “Chomsky Normal Form” – HMU’s own explicit safe-order statement, matching Section 1.4’s opening claim word for word.
6. (SIPSER) §2.1, the proof of Theorem 2.9 (p.107) – Sipser’s own route to the same two simplifications (eliminate ε -rules, then unit rules) on the way to Chomsky normal form; Sipser does not isolate “useless symbols” as its own named step the way HMU does, so Section 1.1 is closer to HMU’s treatment specifically.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 Chomsky and Greibach Normal Forms

Trimming a grammar of useless symbols, ε -productions, and unit productions removes dead weight, but it does not give the surviving rules any *uniform* shape – one production can still be a lone terminal, another a chain of three variables, another a terminal buried between two variables, all coexisting in the same fully simplified grammar. That absence of uniformity is exactly what stands in the way of using a grammar as an efficient algorithm’s input, or as raw material for building an automaton by a fixed recipe: an algorithm that has to handle five unrelated production shapes is five times the algorithm it needs to be. A **normal form** fixes this by restricting every production to one of a small, fixed set of shapes – chosen so that some property of every derivation becomes completely predictable. This document builds the two standard ones: **Chomsky Normal Form (CNF)**, which makes every derivation step binary and is the exact ingredient a polynomial-time membership test needs; and **Greibach Normal Form (GNF)**, which makes every derivation step emit exactly one terminal, and is the exact ingredient a pushdown automaton with no ε -moves needs. Both are solved in full on more than one grammar below.

1.1 Why Simplification Isn’t Enough

Take a grammar already stripped of every useless symbol, every ε -production, and every unit production:

$$E \rightarrow E + T \mid T * F \mid (E) \mid a, \quad T \rightarrow T * F \mid (E) \mid a, \quad F \rightarrow (E) \mid a.$$

By every definition met so far, this grammar is completely simplified. Yet watch what happens to two different strings it generates, both of length 3:

$$E \Rightarrow (E) \Rightarrow (a) \quad - \text{two steps, string } (a),$$

$$E \Rightarrow E + T \Rightarrow a + T \Rightarrow a + a \quad - \text{three steps, string } a + a.$$

Same grammar, same string length, different number of derivation steps – there is no formula relating $|w|$ to the length of w ’s derivation, because nothing in ”simplified” ever promised one. Any algorithm that wants to decide membership by searching through derivations needs exactly such a formula, or it has no way to know when to stop searching.

Normal form

A **normal form** for context-free grammars is a restriction on the shape of every production such that (i) every CFG generating a language without ε has an equivalent grammar obeying the restriction, and (ii) the restriction is chosen so that some feature of every derivation becomes uniform and predictable. Two standard choices:

- **Chomsky Normal Form:** every production is $A \rightarrow BC$ or $A \rightarrow a$. Every step is binary, so a derivation of a string of length n takes *exactly* $2n - 1$ steps, always.
- **Greibach Normal Form:** every production is $A \rightarrow a\alpha$, with α a (possibly empty) string of variables. Every step emits exactly one terminal, so a derivation of a string of length n takes *exactly* n steps, always.
- Both allow the single exception $S \rightarrow \varepsilon$, and only if S never appears on the right of any production – the same carve-out made when ε -productions were first eliminated.

Exercises: Why Normal Forms

1. Find a third string of length 3 generated by this subsection's grammar and count its derivation steps. (Hint: try $a * a$.) Does it match either count above, or give a third one?
2. An algorithm decides membership by trying every derivation up to some length k and checking whether any of them produces w . Without a fixed formula relating derivation length to $|w|$, what value would k have to be to guarantee correctness – and why is that a problem?

1.2 Chomsky Normal Form

CNF's payoff is concrete: given a CNF grammar G and a string w of length n , a dynamic-programming table can decide $w \in L(G)$ in $O(n^3)$ time, by computing, for every substring $w_{i,j}$, exactly which variables can derive it – a computation that only makes sense because CNF guarantees every derivation step either splits a substring into two independently solved halves ($A \rightarrow BC$) or bottoms out at a single terminal ($A \rightarrow a$). This is the **Cocke-Younger-Kasami (CYK)** algorithm.

Discovered three times over

CYK has three independent origins. J. Cocke's version circulated privately and was never published. T. Kasami's rendition appeared only in an internal U.S. Air Force memorandum. D. H. Younger's is the one that reached print, in *Information and Control* in 1967 – which is why the algorithm both is, and is not, quite fairly named after all three.

Chomsky Normal Form

A grammar $G = (V, \Sigma, P, S)$ is in **Chomsky Normal Form** if every production has the shape $A \rightarrow BC$ (with $B, C \in V$) or $A \rightarrow a$ (with $a \in \Sigma$), aside from the single possible exception $S \rightarrow \varepsilon$.

Method: converting a simplified grammar to CNF

Start from a grammar already free of useless symbols, ε -productions, and unit productions. Then:

- For every terminal a appearing in a body of length 2 or more, introduce a fresh variable $X_a \rightarrow a$, and replace every such occurrence of a by X_a . Afterward, every body is either a single terminal or two or more variables.
- Break every remaining body $B_1 B_2 \cdots B_k$ with $k \geq 3$ into a chain: introduce fresh variables $D_1, \dots, D_{k-2}$ and replace the production by $A \rightarrow B_1 D_1$, $D_1 \rightarrow B_2 D_2$, $\dots$, $D_{k-2} \rightarrow B_{k-1} B_k$. If the same long body appears in more than one production, the same chain of D 's can be reused for all of them.

Worked example 1. Apply this to the grammar from Section 1.1. Step (a): the terminals $+, *, (,)$ each appear mixed into a longer body (but a never does – it only ever stands alone), so four new variables suffice:

$$P \rightarrow +, \quad M \rightarrow *, \quad L \rightarrow (, \quad R \rightarrow).$$

Rewriting every mixed body with these:

$$E \rightarrow EPT \mid TMF \mid LER \mid a, \quad T \rightarrow TMF \mid LER \mid a, \quad F \rightarrow LER \mid a.$$

Step (b): three distinct length-3 bodies need binarizing – EPT (only in E), TMF (in E and T), and LER (in E , T , and F) – so three helper variables, each shared across every production that uses that exact tail:

$$C_1 \rightarrow PT, \quad C_2 \rightarrow MF, \quad C_3 \rightarrow ER.$$

The finished CNF grammar:

$$\begin{aligned} E &\rightarrow EC_1 \mid TC_2 \mid LC_3 \mid a, & T &\rightarrow TC_2 \mid LC_3 \mid a, & F &\rightarrow LC_3 \mid a, \\ C_1 &\rightarrow PT, & C_2 &\rightarrow MF, & C_3 &\rightarrow ER, & P &\rightarrow +, & M &\rightarrow *, & L &\rightarrow (, & R &\rightarrow). \end{aligned}$$

Every one of these ten variables now has only $A \rightarrow BC$ or $A \rightarrow a$ productions. As promised, (a) now takes $2(3) - 1 = 5$ steps, and so does $a + a$: check $E \Rightarrow EC_1 \Rightarrow aC_1 \Rightarrow aPT \Rightarrow a+T \Rightarrow a+a$ – five steps, matching the formula exactly, where the original grammar gave three.

Worked example 2. Bolt one more layer onto the same grammar – an identifier variable I for letters and digits, sitting under F – to get, after simplification,

$$\begin{aligned} E &\rightarrow E+T \mid T*F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid IO \mid I1, \\ T &\rightarrow T*F \mid (E) \mid a \mid b \mid Ia \mid Ib \mid IO \mid I1, \quad F \rightarrow (E) \mid a \mid b \mid Ia \mid Ib \mid IO \mid I1, \\ I &\rightarrow a \mid b \mid Ia \mid Ib \mid IO \mid I1. \end{aligned}$$

Now a and b *do* appear mixed into longer bodies (inside Ia , Ib), alongside the same four operator/parenthesis terminals as before – eight terminals in total, needing eight new variables. $C_1, C_2, C_3, P, M, L, R$ carry over completely unchanged from Worked Example 1, since E, T , and F 's $+, *, (,)$ -bodies are identical; only four more are new, one per digit/letter terminal:

$$A \rightarrow a, \quad B \rightarrow b, \quad Z \rightarrow 0, \quad O \rightarrow 1.$$

Step (a) rewrites the new I -bodies as $IA \mid IB \mid IZ \mid IO$ (already binary, no step (b) needed for these). The finished CNF grammar:

$$\begin{aligned} E &\rightarrow EC_1 \mid TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO, \\ T &\rightarrow TC_2 \mid LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO, \quad F \rightarrow LC_3 \mid a \mid b \mid IA \mid IB \mid IZ \mid IO, \\ I &\rightarrow a \mid b \mid IA \mid IB \mid IZ \mid IO, \\ C_1 &\rightarrow PT, \quad C_2 \rightarrow MF, \quad C_3 \rightarrow ER, \quad A \rightarrow a, \quad B \rightarrow b, \quad Z \rightarrow 0, \quad O \rightarrow 1, \\ P &\rightarrow +, \quad M \rightarrow *, \quad L \rightarrow (, \quad R \rightarrow). \end{aligned}$$

This is Hopcroft, Motwani, and Ullman's own running example for the construction: the same mechanical two steps, just applied to a grammar with more terminals to account for.

Exercises: Chomsky Normal Form

1. Convert $S \rightarrow AB \mid a, A \rightarrow aA \mid a, B \rightarrow bB \mid b$ to CNF.
2. Convert $S \rightarrow aSb \mid ab$ (generating $\{a^n b^n : n \geq 1\}$) to CNF. (Hint: both productions have mixed terminal/variable bodies – step (a) is needed before step (b) can even begin.)
3. Convert $S \rightarrow aA \mid \varepsilon, A \rightarrow aA \mid bB \mid b, B \rightarrow b$ to CNF. Check first that $S \rightarrow \varepsilon$ is actually legal to keep here under the exception in the definition box – does S appear on the right of any production?

1.3 Greibach Normal Form

CNF's binary steps are perfect for a table-filling algorithm that does not care what order it fills the table in. A pushdown automaton reading input left to right cares a great deal: to build one directly from a grammar with no ε -moves at all – one input symbol consumed per move, always – what is needed is a grammar where every production already emits exactly one terminal per step, with the terminal read *first*.

Greibach Normal Form

A grammar $G = (V, \Sigma, P, S)$ is in **Greibach Normal Form** if every production has the shape $A \rightarrow a\alpha$, with $a \in \Sigma$ a single terminal and $\alpha \in V^*$ a (possibly empty) string of variables, aside from the single possible exception $S \rightarrow \varepsilon$.

Every terminal-first production, read as a top-down parsing step, tells you which rule to try next from the very next input symbol alone – no guessing, no lookahead beyond one symbol. That same terminal-first shape is exactly what lets a pushdown automaton push α onto its stack and consume a in a single, ε -free move. The whole construction rests on two supporting lemmas, used repeatedly rather than just once each.

Lemma 1: substitution

If $A \rightarrow B\alpha$ is a production and $B \rightarrow \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n$ are *all* of B 's productions (α, β_i any strings of variables and/or terminals – either can be anything), then replacing $A \rightarrow B\alpha$ by

$$A \rightarrow \beta_1\alpha \mid \beta_2\alpha \mid \cdots \mid \beta_n\alpha$$

leaves the grammar's language unchanged: every way of expanding B first and then continuing with α is already accounted for on the right.

Lemma 2: eliminating immediate left recursion

Given $A \rightarrow A\alpha \mid \beta$ (α, β any strings, β not itself starting with A), introduce a fresh variable A' and write

$$A \rightarrow \beta A', \quad A' \rightarrow \alpha A' \mid \varepsilon.$$

This visibly generates the same language – A' just spells out “zero or more repetitions of α ,” exactly what the left-recursive rule $A \rightarrow A\alpha$ was doing. Removing the one resulting ε -production from A' , the same way any ε -production is removed, gives the equivalent, ε -free pair

$$A \rightarrow \beta \mid \beta A', \quad A' \rightarrow \alpha \mid \alpha A'.$$

Generalized to several recursive and non-recursive alternatives at once – $A \rightarrow A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_k$ – this becomes

$$A \rightarrow \beta_1 \mid \cdots \mid \beta_k \mid \beta_1 A' \mid \cdots \mid \beta_k A', \quad A' \rightarrow \alpha_1 \mid \cdots \mid \alpha_m \mid \alpha_1 A' \mid \cdots \mid \alpha_m A'.$$

Method: converting to GNF

Order the variables $A_1, \dots, A_n$ *arbitrarily* – literally any fixed numbering works, because nothing below depends on which variable ends up called A_1 versus A_n , only on having picked *some* order and sticking to it throughout. Then:

1. For $i = 1$ to n : while some production $A_i \rightarrow A_j \alpha$ has $j < i$, apply **Lemma 1** to substitute every one of A_j 's current bodies in place of that leading A_j . Repeat until every A_i -body starts with a terminal or with some A_k , $k > i$.
2. If this ever leaves $A_i \rightarrow A_i \alpha$ (immediate left recursion) among A_i 's bodies, apply **Lemma 2** (its generalized form) directly to A_i 's productions – writing Z_i for the fresh variable Lemma 2 calls A' , to keep track of which A_i it belongs to.
3. Finally, working from the highest-indexed variable down, substitute every remaining variable-led body (which, by step 1, leads with a strictly higher index) using that variable's own now-finished, terminal-led bodies – including for every Z_i introduced in step 2, whose bodies always lead with some A_j .

Step 1 eliminates *every* backward reference under *any* numbering, so no ordering is ever privileged the way it was for eliminating ε -productions and unit productions before useless symbols, where getting the sequence wrong left the wrong answer behind. Here, a different numbering can only change which variables end up doing the substitution work and how large the final grammar turns out to be – never whether the construction succeeds.

This is the rigorous construction Hopcroft, Motwani, and Ullman set as an exercise, credited there to M. C. Paull; their version specializes it to start from a grammar already in CNF, which step 1 alone is then enough to handle.

Worked example 1 (no recursion to eliminate). $S \rightarrow XY \mid a$, $X \rightarrow aX \mid bY$, $Y \rightarrow b$. Name the variables exactly the way the method itself does, in this order: $A_1 = S$, $A_2 = X$, $A_3 = Y$, so the grammar to feed the method is

$$A_1 \rightarrow A_2 A_3 \mid a, \quad A_2 \rightarrow a A_2 \mid b A_3, \quad A_3 \rightarrow b.$$

Step 1: A_2 and A_3 are already fully terminal-led – nothing to do for either. Only $A_1 \rightarrow A_2 A_3$ is variable-led, with A_2 (a higher index) leading, so it is deferred to step 3: substitute A_2 's two bodies in for the leading A_2 :

$$A_1 \rightarrow A_2 A_3 \implies A_1 \rightarrow (a A_2 \mid b A_3) A_3 = a A_2 A_3 \mid b A_3 A_3.$$

Together with the untouched $A_1 \rightarrow a$, the method's raw output is

$$A_1 \rightarrow a A_2 A_3 \mid b A_3 A_3 \mid a, \quad A_2 \rightarrow a A_2 \mid b A_3, \quad A_3 \rightarrow b.$$

Resubstituting the original names ($A_1 = S$, $A_2 = X$, $A_3 = Y$) gives the finished GNF grammar:

$$S \rightarrow aXY \mid bYY \mid a, \quad X \rightarrow aX \mid bY, \quad Y \rightarrow b.$$

No left recursion ever appeared, because no variable here could reach a sentential form starting with itself, directly or by substitution.

Worked example 2 (indirect, then immediate, left recursion). $S \rightarrow AA \mid 0$, $A \rightarrow SS \mid 1$ – this is Hopcroft, Motwani, and Ullman’s own exercise, and the grammar is already in CNF. The grammar’s own second variable already happens to be called A , which would collide with the method’s own generic A_i notation, so the renaming step earns its keep here: $A_1 = S$, $A_2 = A$, giving

$$A_1 \rightarrow A_2A_2 \mid 0, \quad A_2 \rightarrow A_1A_1 \mid 1.$$

Step 1, $i = 2$: the leading A_1 in $A_2 \rightarrow A_1A_1$ has index $1 < 2$, so substitute A_1 ’s bodies ($A_2A_2 \mid 0$) for it:

$$A_2 \rightarrow A_1A_1 \mid 1 \implies A_2 \rightarrow (A_2A_2 \mid 0)A_1 \mid 1 = A_2A_2A_1 \mid 0A_1 \mid 1.$$

Now $A_2A_2A_1$ leads with A_2 itself – immediate left recursion, $\alpha = A_2A_1$, with β ’s $0A_1$ and 1 . Step 2 introduces Z_2 :

$$A_2 \rightarrow 0A_1 \mid 1 \mid 0A_1Z_2 \mid 1Z_2, \quad Z_2 \rightarrow A_2A_1 \mid A_2A_1Z_2.$$

A_2 is now fully terminal-led. Back to $i = 1$: $A_1 \rightarrow A_2A_2 \mid 0$ has its leading A_2 at a higher index, deferred to step 3, which substitutes A_2 ’s finished bodies into A_1 , then into Z_2 :

$$\begin{aligned} A_1 \rightarrow A_2A_2 \mid 0 &\implies A_1 \rightarrow 0A_1A_2 \mid 1A_2 \mid 0A_1Z_2A_2 \mid 1Z_2A_2 \mid 0, \\ Z_2 \rightarrow A_2A_1 \mid A_2A_1Z_2 &\implies Z_2 \rightarrow 0A_1A_1 \mid 1A_1 \mid 0A_1Z_2A_1 \mid 1Z_2A_1 \\ &\quad \mid 0A_1A_1Z_2 \mid 1A_1Z_2 \mid 0A_1Z_2A_1Z_2 \mid 1Z_2A_1Z_2. \end{aligned}$$

Resubstituting the original names ($A_1 = S$, $A_2 = A$, and writing Z for the freshly introduced Z_2 , which has no counterpart in the original grammar) gives the finished GNF grammar:

$$\begin{aligned} S &\rightarrow 0SA \mid 1A \mid 0SZA \mid 1ZA \mid 0, \quad A \rightarrow 0S \mid 1 \mid 0SZ \mid 1Z, \\ Z &\rightarrow 0SS \mid 1S \mid 0SZS \mid 1ZS \mid 0SSZ \mid 1SZ \mid 0SZSZ \mid 1ZSZ. \end{aligned}$$

This has three variables and seventeen productions – against the original’s two variables and four. As a check, the original grammar derives 11 (the string, in three steps: $S \Rightarrow AA \Rightarrow 1A \Rightarrow 11$), while the GNF grammar derives it in exactly two, matching its own length: $S \Rightarrow 1A \Rightarrow 11$.

Worked example 3 (same shape, already named A_1, A_2). $A_1 \rightarrow A_2A_2 \mid a$, $A_2 \rightarrow A_1A_1 \mid b$ – the same indirect-then-immediate pattern as Worked Example 2, with a, b in place of $0, 1$, except this time the variables already carry the method’s own names, so there is no renaming step at all. Step 1, $i = 2$: the leading A_1 in $A_2 \rightarrow A_1A_1$ has index $1 < 2$, so Lemma 1 substitutes A_1 ’s bodies ($A_2A_2 \mid a$) for it:

$$A_2 \rightarrow A_1A_1 \mid b \implies A_2 \rightarrow (A_2A_2 \mid a)A_1 \mid b = A_2A_2A_1 \mid aA_1 \mid b.$$

$A_2A_2A_1$ leads with A_2 itself – immediate left recursion, $\alpha = A_2A_1$, β ’s aA_1 and b . Lemma 2 introduces Z_2 :

$$A_2 \rightarrow aA_1 \mid b \mid aA_1Z_2 \mid bZ_2, \quad Z_2 \rightarrow A_2A_1 \mid A_2A_1Z_2.$$

A_2 is now fully terminal-led. Back to $i = 1$: $A_1 \rightarrow A_2A_2 \mid a$ has its leading A_2 at a higher index, deferred to step 3, which substitutes A_2 's finished bodies into A_1 , then into Z_2 :

$$\begin{aligned} A_1 \rightarrow A_2A_2 \mid a &\implies A_1 \rightarrow aA_1A_2 \mid bA_2 \mid aA_1Z_2A_2 \mid bZ_2A_2 \mid a, \\ Z_2 \rightarrow A_2A_1 \mid A_2A_1Z_2 &\implies Z_2 \rightarrow aA_1A_1 \mid bA_1 \mid aA_1Z_2A_1 \mid bZ_2A_1 \\ &\quad \mid aA_1A_1Z_2 \mid bA_1Z_2 \mid aA_1Z_2A_1Z_2 \mid bZ_2A_1Z_2. \end{aligned}$$

Since A_1, A_2 were already the grammar's own names, there is nothing left to resubstitute – this *is* the finished GNF grammar, with Z_2 (written Z) the only genuinely new variable. As a check, $A_1 \Rightarrow bA_2 \Rightarrow bb$ derives bb in exactly two steps, matching its length, against three steps ($A_1 \Rightarrow A_2A_2 \Rightarrow bA_2 \Rightarrow bb$) in the original.

Worked example 4 (four variables, one immediate recursion). $S \rightarrow XA \mid BB$, $B \rightarrow b \mid SB$, $X \rightarrow b$, $A \rightarrow a$. Name $A_1 = S$, $A_2 = X$, $A_3 = A$, $A_4 = B$:

$$A_1 \rightarrow A_2A_3 \mid A_4A_4, \quad A_2 \rightarrow b, \quad A_3 \rightarrow a, \quad A_4 \rightarrow b \mid A_1A_4.$$

Step 1: A_2 and A_3 are already terminal-led. For $i = 4$ ($A_4 \rightarrow b \mid A_1A_4$), the leading A_1 has index $1 < 4$: Lemma 1 substitutes A_1 's bodies ($A_2A_3 \mid A_4A_4$) for it,

$$A_4 \rightarrow b \mid A_1A_4 \implies A_4 \rightarrow b \mid A_2A_3A_4 \mid A_4A_4A_4,$$

and now $A_2A_3A_4$ leads with A_2 , still index $2 < 4$: Lemma 1 again, substituting A_2 's finished body b ,

$$A_4 \rightarrow b \mid bA_3A_4 \mid A_4A_4A_4.$$

$A_4A_4A_4$ leads with A_4 itself – immediate left recursion, $\alpha = A_4A_4$, β 's b and bA_3A_4 . Lemma 2 introduces Z_4 :

$$A_4 \rightarrow b \mid bA_3A_4 \mid bZ_4 \mid bA_3A_4Z_4, \quad Z_4 \rightarrow A_4A_4 \mid A_4A_4Z_4.$$

A_4 is now fully terminal-led, and $A_1 \rightarrow A_2A_3 \mid A_4A_4$ was already deferred (both A_2 and A_4 are higher-indexed). Step 3 substitutes A_2 's and A_4 's finished bodies into A_1 , then A_4 's into Z_4 :

$$\begin{aligned} A_1 \rightarrow A_2A_3 \mid A_4A_4 &\implies A_1 \rightarrow bA_3 \mid bA_3A_4A_4 \mid bA_4 \mid bA_3A_4Z_4A_4 \mid bZ_4A_4, \\ Z_4 \rightarrow A_4A_4 \mid A_4A_4Z_4 &\implies Z_4 \rightarrow bA_4 \mid bA_3A_4A_4 \mid bZ_4A_4 \mid bA_3A_4Z_4A_4 \\ &\quad \mid bA_4Z_4 \mid bA_3A_4A_4Z_4 \mid bZ_4A_4Z_4 \mid bA_3A_4Z_4A_4Z_4. \end{aligned}$$

Resubstituting the original names ($A_1 = S$, $A_2 = X$, $A_3 = A$, $A_4 = B$, writing Z for Z_4) gives the finished GNF grammar:

$$\begin{aligned} S &\rightarrow bA \mid bABB \mid bB \mid bABZB \mid bZB, & X &\rightarrow b, & A &\rightarrow a, \\ B &\rightarrow b \mid bAB \mid bZ \mid bABZ, \\ Z &\rightarrow bB \mid bABB \mid bZB \mid bABZB \mid bBZ \mid bABBZ \mid bZBZ \mid bABZBZ. \end{aligned}$$

This four-variable, six-production original grows to five variables and nineteen productions – the biggest blow-up of any example in this document, from a grammar with two separate occurrences of B on the right of $S \rightarrow BB$, each one independently triggering the substitution machinery above.

To think!!

CNF's construction grows the grammar by roughly one new variable per excess symbol in a body – linear in the damage it is repairing. GNF's construction, in Worked Example 2 above, took a four-production grammar to seventeen, and Z alone carries eight alternatives that did not exist before. For which of this document's two headline use cases – the $O(n^3)$ CYK membership test, or building an ε -free pushdown automaton – does that blow-up in production count actually cost something, and for which is it beside the point?

Exercises: Greibach Normal Form

1. Convert $S \rightarrow AB$, $A \rightarrow a$, $B \rightarrow SB \mid b$ to GNF. (Hint: after conversion, check whether every variable is still reachable from S – substitution can strand a variable the same way eliminating a unit production can.)
2. Convert $S \rightarrow XY$, $X \rightarrow b$, $Y \rightarrow YX \mid a$ to GNF. (Hint: unlike both worked examples above, the left recursion here is immediate from the start – no substitution is needed before step 2 applies.)
3. Worked Example 1 needed no Z -style helper variable at all; Worked Example 2 needed one. What is it about a grammar where no variable can reach a sentential form starting with itself – directly, or after substituting other variables in – that guarantees step 2 of the method is never triggered?
4. Redo Worked Example 2 with the *opposite* ordering, A before S . (Hint: this time it is S , not A , that develops the immediate left recursion, and needs its own Z .) Confirm your grammar still derives 11 in exactly two steps, and compare its production count against the original ordering's seventeen.

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (HMU) §7.1.5 “Chomsky Normal Form” (p.272) – the formal definition and two-part construction algorithm, matching Section 1.2’s method box, together with Example 7.15 (the same identifier-augmented grammar solved as this document’s Worked Example 2).
2. (HMU) §7.4.4 “Testing Membership in a CFL” (p.303) – the Cocke-Younger-Kasami algorithm, CNF’s payoff use case named at the top of Section 1.2.
3. (HMU) §7.1, the unnumbered “Greibach Normal Form” aside (p.277) – HMU’s own brief, no-proof statement of GNF and its consequence for ε -free pushdown automata, matching Section 1.3’s opening.
4. (HMU) Exercises 7.1.11–7.1.12 (pp.278–279) – the rigorous GNF construction (credited there to M. C. Paull) and the grammar $S \rightarrow AA \mid 0$, $A \rightarrow SS \mid 1$, solved in full as Section 1.3’s Worked Example 2.
5. (SIPSER) §2.1, the proof of Theorem 2.9 (p.107) – Sipser’s own CNF construction (add a new start variable, eliminate ε -rules and unit rules, then convert the rest), matching Section 1.2. Sipser’s textbook does not cover Greibach Normal Form at all, so Section 1.3 has no Sipser counterpart to point to.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 Pushdown Automata

A finite automaton recognizes a language by remembering only a bounded amount of information – a state, and nothing else – which is exactly why no finite automaton can recognize $\{0^n 1^n : n > 0\}$: checking that the count of 1's matches the count of 0's needs a count that grows without bound, and a finite set of states can never hold one. Context-free languages need more memory than that, but not arbitrary memory – memory restricted, very specifically, to a **stack**: a store that can only be read or changed at its top, one symbol at a time. A **pushdown automaton (PDA)** bolts exactly this kind of stack onto a finite automaton's finite control, and turns out to recognize exactly the context-free languages, no more and no less. This document builds the model: the stack and the finite control as two distinct kinds of memory, the formal six-tuple definition, the six meaningful shapes a single transition can take, and the two bookkeeping devices – configurations and the *yields* relation – precise enough to define acceptance itself. As defined below, a PDA is **nondeterministic**: at a given configuration it may have several legal moves on offer at once, exploring all of them the way an NFA explores several states at once, and needs only one of them to lead to acceptance. A **deterministic pushdown automaton (DPDA)** is a separate, more restrictive model, tied closely to how real parsers work; this document works only with the nondeterministic kind.

1.1 Finite Control and the Stack

A PDA's finite control is exactly a finite automaton's set of states – it remembers *which* of finitely many situations it is in, and nothing more. The stack is what makes the difference: an unbounded pile of symbols, but accessible only at the top. At each move, a PDA may **push** one new symbol onto the stack, **pop** the top symbol off, or leave the stack untouched – never reach in and inspect or change anything buried underneath without first removing everything sitting above it.

That one restriction is already enough to count. Take $L = \{0^n 1^n : n > 0\}$: push a marker onto the stack for every 0 read, then pop one marker for every 1 read. If the stack empties out at the exact moment the input runs out, every 0 was matched by a 1 and the string is accepted; if the stack empties early, or the input runs out with markers still stacked up, it is rejected. Nothing about this procedure needs to know n in advance, and nothing about it needs more than one symbol of memory at a time – just a pile of them.

1.2 The Formal Definition: A Six-Tuple

Pushdown automaton

A **pushdown automaton** is a six-tuple $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$:

- Q – a finite set of **states** (the finite control);
- Σ – a finite **input alphabet**;
- Γ – a finite **stack alphabet** – not necessarily the same symbols as Σ ; a PDA is free to push bookkeeping symbols (a bottom-of-stack marker, say) that never appear in the input at all;
- $\delta : Q \times \Sigma_\epsilon \times \Gamma_\epsilon \rightarrow \mathcal{P}(Q \times \Gamma_\epsilon)$ – the **transition function**, where $\Sigma_\epsilon = \Sigma \cup \{\epsilon\}$ and $\Gamma_\epsilon = \Gamma \cup \{\epsilon\}$;
- $q_0 \in Q$ – the **start state**;
- $F \subseteq Q$ – the set of **accepting states**.

δ mapping to a *set* of (state, stack symbol) pairs, rather than to a single one, is precisely what makes this a *nondeterministic* model – more than one legal move can be on offer from the same state, reading the same symbol, seeing the same stack top.

1.3 Reading a Transition: The Six Cases

In practice a PDA is drawn, not tabulated: an arc from state q to state r , labeled $a, X \rightarrow Y$.

Reading a transition in the state diagram

$$q \xrightarrow{a, X \rightarrow Y} r$$

reads as: “from state q , read a and the symbol X off the top of the stack, then move to r and replace X with Y .” Because the automaton is nondeterministic, several arcs – even with different labels, even leading to different states – can apply to the very same configuration at once, and the PDA is understood to explore every one of them in parallel, the same way an NFA explores several states at once on the same input.

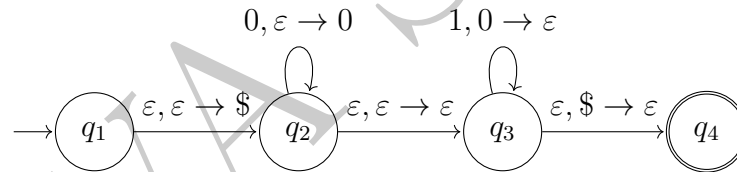
Both a and X , and separately Y , may themselves be ϵ , giving six meaningful combinations of “what is read,” “what the stack top must show,” and “what gets pushed back”:

The six cases of a transition

1. $\varepsilon, \varepsilon \rightarrow Y$ **just push** Y – no input read, nothing required on top
2. $\varepsilon, X \rightarrow \varepsilon$ **just pop** – fires only if the top is exactly X
3. $a, \varepsilon \rightarrow \varepsilon$ **read only** – no push, no pop
4. $a, \varepsilon \rightarrow Y$ **read, then push** Y – no pop
5. $a, X \rightarrow \varepsilon$ **read, then pop** – fires only if the top is exactly X
6. $a, X \rightarrow Y$ **read, then replace** X by Y – pop X , push Y ; fires only if the top is X

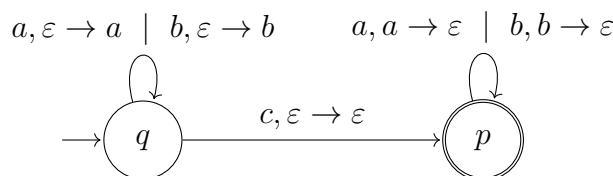
A seventh combination, $\varepsilon, \varepsilon \rightarrow \varepsilon$ – change state, read nothing, touch nothing on the stack – is left off this list because on its own it does nothing at all; used sparingly, though, it is exactly the move a PDA needs to **nondeterministically guess** something about input it cannot yet see, as Worked Example 3 below will need.

Worked example 1: $\{0^n 1^n : n > 0\}$. Formalizing Section 1.1's narrative, with \$ a stack symbol reserved to mark the bottom of the stack:



$q_1 \rightarrow q_2$ is case 1 (just push \$, marking the bottom before anything real goes on top). q_2 's self-loop is case 4 (read a 0, push it). $q_2 \rightarrow q_3$ is the unlisted seventh case – here it does nothing *except* switch from pushing to popping, and needs no guess, since every 0 has to come before every 1 in this language. q_3 's self-loop is case 5 (read a 1, and it only fires if a 0 is sitting on top to pop). $q_3 \rightarrow q_4$ is case 2 (just pop) – and it can only fire once every real 0 has been popped back off and \$ is exposed on top, which is exactly the signal this PDA needs to know it is safe to accept.

Worked example 2: $\{wcw^R : w \in \{a,b\}^*\}$. A center marker c tells this PDA exactly where to stop pushing and start popping – no guessing, and no \$ needed at all:



q 's self-loop is case 4 twice over (read a or b , push what was just read). $q \rightarrow p$ is case 3 (read the marker c , touch nothing on the stack). p 's self-loop is case 6 twice over (read a or

b , and only pop if the matching symbol is on top – a mismatch simply gets stuck, with no legal move to take). On $abcba$: push a , push b , read c and switch to p , read b and pop the matching b , read a and pop the matching a – input exhausted, stack empty, in the accepting state p . On $abcc$: push a , push b , read c and switch to p , and then the second c has no matching arc out of p at all – stuck, rejected.

1.4 Configurations and the Yields Relation

A single triple records everything about a PDA mid-computation.

Configuration (instantaneous description) and the yields relation

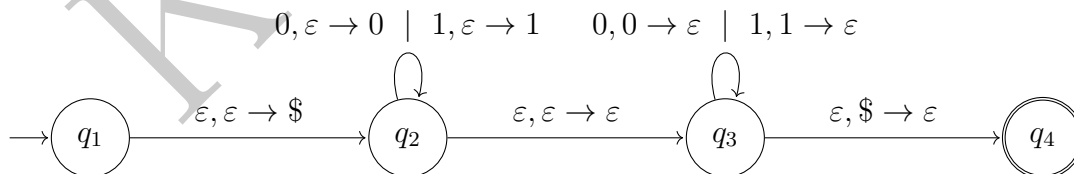
A **configuration** of $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ is a triple $(q, w, \alpha) \in Q \times \Sigma^* \times \Gamma^*$: the current state q , the unread remainder w of the input, and the current stack contents α – written left to right, with the *rightmost* symbol of α the top of the stack.

$(q, w, \alpha) \vdash (r, w', \alpha')$ (“**yields** in one step”) if there is some $a \in \Sigma_\varepsilon$, $X \in \Gamma_\varepsilon$, $Y \in \Gamma_\varepsilon$ with $(r, Y) \in \delta(q, a, X)$, such that:

- $w = aw'$ if $a \neq \varepsilon$ (the symbol a is consumed), or $w' = w$ if $a = \varepsilon$ (nothing is consumed);
- $\alpha = \beta X$ and $\alpha' = \beta Y$ for some $\beta \in \Gamma^*$, if $X \neq \varepsilon$ (the top symbol X is popped and Y is pushed in its place); or $\alpha' = \alpha Y$ if $X = \varepsilon$ (nothing is popped – Y is simply pushed on top of whatever was already there).

$\vdash^*$ is the reflexive-transitive closure of $\vdash$: zero or more steps.

Worked example 3: $\{ww^R : w \in \{0,1\}^*\}$. With no marker in the middle of a string like 001100 to say where w ends and w^R begins, this PDA has to *nondeterministically guess* that point – exactly the unlisted seventh case from Section 1.3:



Here $\$$ earns its keep beyond bookkeeping: if $q_2 \rightarrow q_3$ guesses the middle *wrong*, popping in q_3 either runs out of matching symbols early or leaves real symbols stranded above $\$$, and $q_3 \rightarrow q_4$ – which only fires when $\$$ itself is exposed on top – simply never becomes available. Wrong guesses get stuck; nondeterminism only needs the one correct guess to succeed. Tracing the correct guess on $w = 001$ (so $ww^R = 001100$), continuing the two steps already familiar from class:

$$(q_1, 001100, \varepsilon) \vdash (q_2, 001100, \$) \vdash (q_2, 01100, \$0) \vdash (q_2, 1100, \$00) \vdash (q_2, 100, \$001)$$

$$\vdash (q_3, 100, \$001) \vdash (q_3, 00, \$00) \vdash (q_3, 0, \$0) \vdash (q_3, \varepsilon, \$) \vdash (q_4, \varepsilon, \varepsilon).$$

Nine steps, ending with the input exhausted, the stack empty, and $q_4 \in F$ – an accepting computation.

Acceptance

For $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$, two conventions, both starting every computation at (q_0, w, ε) – empty stack, all of w still unread:

- **Acceptance by final state:**

$$L(P) = \{w \in \Sigma^* : (q_0, w, \varepsilon) \vdash^* (q, \varepsilon, \alpha) \text{ for some } q \in F, \alpha \in \Gamma^*\}$$

– the state reached must be accepting; the stack’s final contents are irrelevant.

- **Acceptance by empty stack:**

$$N(P) = \{w \in \Sigma^* : (q_0, w, \varepsilon) \vdash^* (q, \varepsilon, \varepsilon) \text{ for some } q \in Q\}$$

– the stack must end empty; the state reached is irrelevant, and F can even be left empty entirely.

Every worked example above happened to satisfy both conventions at once, but that is a property of how those particular PDAs were built, not a general law: a short, standard construction converts any PDA using one convention into an equivalent PDA using the other, and the two conventions define exactly the same class of languages between them.

Exercises: Constructing PDAs

1. Balanced parentheses. Construct a PDA (accepting by either convention – your choice) for the language of balanced parentheses over $\{(,)\}$. (Hint: this needs only one stack symbol besides \$, and no guessing at all – every (pushes, every) pops.)
2. Equal numbers of a ’s and b ’s. Construct a PDA for $\{w \in \{a, b\}^* : \#_a(w) = \#_b(w)\}$, where a ’s and b ’s may appear in *any* order, not just grouped together. (Hint: a single stack symbol can represent either “one extra a ” or “one extra b ,” depending on what is currently on top when the next letter arrives – work out all six cases from Section 1.3 this needs before drawing a single arc.)
3. $\{a^i b^j c^k : i = j \text{ or } j = k\}$. Construct a PDA for this language. (Hint: the “or,” not the counting, is the hard part – nondeterminism lets a PDA *guess*, right at the start, which of the two disjuncts it is going to verify, and commit to checking only that one for the rest of the string.)

1.5 Deterministic Pushdown Automata

An NFA and a DFA recognize exactly the same languages – the subset construction turns any NFA into an equivalent DFA, so nondeterminism there buys convenience, never extra power. The same question for pushdown automata has a different answer.

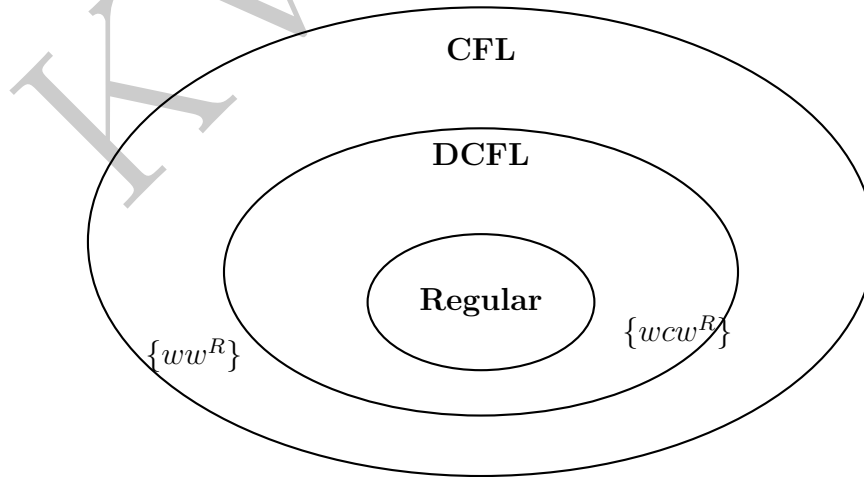
Deterministic pushdown automaton

A PDA $P = (Q, \Sigma, \Gamma, \delta, q_0, F)$ is **deterministic** (a **DPDA**) if, for every $q \in Q$ and $X \in \Gamma_\varepsilon$:

- $|\delta(q, a, X)| \leq 1$ for every $a \in \Sigma_\varepsilon$ – at most one legal move per combination of state, input symbol (or ε), and stack top;
- whenever $\delta(q, \varepsilon, X) \neq \emptyset$, also $\delta(q, a, X) = \emptyset$ for every $a \in \Sigma$ – an available ε -move may never compete with an available real-symbol move reading the same stack top.

Every configuration of a DPDA has at most one configuration it yields to: no branching, no parallel exploration, one single computation to trace.

Unlike DFA and NFA, **a DPDA and a PDA are not equivalent in power** – there is no determinizing construction here, and none can exist. Worked Example 2's $\{wcw^R\}$ PDA already satisfies both conditions above exactly as drawn: no two arcs out of q or out of p ever compete for the same input symbol and stack top, and neither state has an ε -move at all. But Worked Example 3's $\{ww^R\}$ genuinely cannot be done this way – *no* DPDA can recognize it, for precisely the reason its PDA needed the unlisted seventh case: without a marker in the middle, deciding where w ends and w^R begins is a guess, and a DPDA is, by definition, a machine that never guesses.



- **Regular** languages are recognized by a DFA (equivalently, an NFA).
- **DCFLs** (deterministic context-free languages) are recognized by a DPDA.
- **CFLs** are recognized by a PDA – this document's nondeterministic kind.

Both containments are strict. $\{wcw^R\}$ sits in DCFL but not in Regular: no finite amount of state can remember an unboundedly long w . $\{ww^R\}$ sits in CFL but not in DCFL: no DPDA can guess where the middle of the string is.

To think!!

Suppose someone hands you a PDA – a full six-tuple, states and all – and, checking every arc in it, you find that not one of them ever pushes or pops: every single transition has $X = \varepsilon$ and $Y = \varepsilon$, the stack sitting there completely unused from the first move to the last. What does that tell you about the language this PDA recognizes – where does it have to sit in the Euler diagram above, and why? (Hint: strip the stack out of the six-tuple entirely, component by component, and see what is left.)

Exercises: Deterministic Pushdown Automata

1. Go through Worked Example 2's PDA arc by arc and confirm it satisfies both DPDA conditions above – for every state, check that no two outgoing arcs ever compete for the same input symbol (or ε) and the same stack top.
2. Worked Example 1's PDA has an $\varepsilon, \varepsilon \rightarrow \varepsilon$ move out of q_2 available at the same time q_2 still has a real move on 0 – so as *drawn*, it violates the DPDA conditions. Yet $\{0^n 1^n : n > 0\}$ is a well-known DCFL (some DPDA does recognize it). What does that tell you about the difference between “this particular PDA is nondeterministic” and “this language has no DPDA at all”?

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (SIPSER) §2.2 “Pushdown Automata,” Definition 2.13 (p.111) – the six-tuple definition matching Section 1.2 exactly, including the $\delta : Q \times \Sigma_\varepsilon \times \Gamma_\varepsilon \rightarrow \mathcal{P}(Q \times \Gamma_\varepsilon)$ transition function.
2. (HMU) §6.1 “Definition of the Pushdown Automaton” – HMU's alternate seven-tuple $(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, with an explicit initial stack symbol Z_0 instead of Section 1.2's \$-by-convention, and a transition function that may push an entire *string* of stack symbols in one move rather than Section 1.3's single symbol Y at a time.
3. (HMU) §6.1.4 “Instantaneous Descriptions of a PDA” – matches Section 1.4's configuration and $\vdash$ definitions, in HMU's seven-tuple terms.

4. (HMU) §6.2.1 “Acceptance by Final State” and §6.2.2 “Acceptance by Empty Stack” – both acceptance conventions from Section 1.4’s closing box, including the equivalence between them.
5. (HMU) §6.4 “Deterministic Pushdown Automata,” §6.4.1 “Definition of a Deterministic PDA” – the formal conditions matching Section 1.5’s box, and HMU’s own development of why a DPDA and a PDA are not equivalent in power.

4 Disclaimer

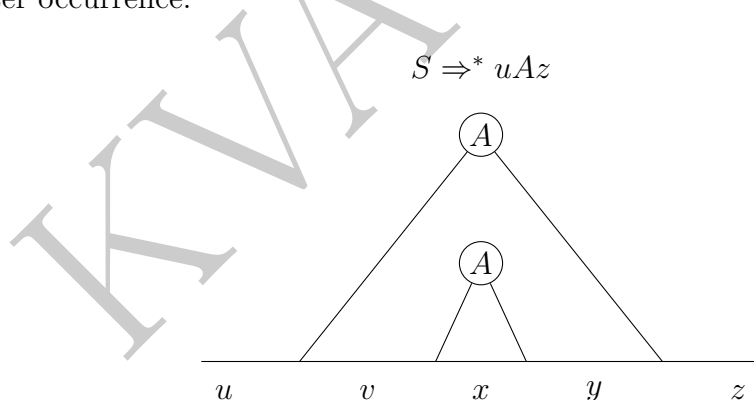
The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.

1 The Pumping Lemma for Context-Free Languages

Every construction met so far proves a language context-free by *exhibiting* something for it – a grammar, a PDA. That only ever runs in one direction: to show a language is *not* context-free, failing to find a grammar proves nothing, since the failure might just be a lack of imagination. What is needed is a property every context-free language is *forced* to have, so that finding one language without that property settles the matter outright. A regular language’s own pumping lemma gets this from one repeated *state* in a long enough run of a DFA; the context-free version below gets it from one repeated *variable*, forced to repeat somewhere along a single root-to-leaf path of a tall enough parse tree. This document proves that fact (Sipser’s Theorem 2.34), then puts it to work the same way every pumping argument gets used in practice: as an adversary game, played out in full on two languages, $\{a^n b^n c^n : n \geq 0\}$ and the *copy language* $\{ww : w \in \Sigma^*\}$.

1.1 Why a Parse Tree, Not a DFA

Take any context-free language A and a grammar G for it already in Chomsky Normal Form, so every internal node of every parse tree has exactly two children. With m the number of variables in G , a tree of height h has a yield of length at most 2^h – each level at most doubles the leaf count. Set $p = 2^m$. Any string $s \in L(G)$ with $|s| \geq p$ therefore needs a parse tree of height strictly greater than m , which means *some* root-to-leaf path in that tree passes through more than m nodes labeled with variables – and since G only has m distinct variables to offer, by pigeonhole some variable A must repeat on that one path, nested inside its own later occurrence.



That repeated A splits s into five pieces, $s = uvxyz$: x is the yield of the *inner* A ’s own subtree, vxy together is the yield of the *outer* A ’s (larger) subtree, and u, z are whatever the rest of the tree contributes on either side. Because both subtrees are valid derivations of the very same variable A , either one can be substituted for the other: replace the outer subtree’s copy of A with the inner one instead of its own contents, and A still legally derives vxy – so it can just as well derive x alone (pumping down, $i = 0$), or $v(vxy)y = v^2xy^2$ inserted in its place (pumping up, $i = 2$), or that trick repeated i times for any $i \geq 0$. Every one of these substitutions still derives from S down through $u, \dots, z$ exactly as before, so $uv^i xy^i z \in L(G)$ for every $i \geq 0$.

The pumping lemma for context-free languages

If A is a context-free language, there exists a **pumping length** $p \geq 1$ such that every $s \in A$ with $|s| \geq p$ can be written $s = uvxyz$ with:

1. $uv^i xy^i z \in A$ for every $i \geq 0$,
2. $|vy| > 0$,
3. $|vxy| \leq p$.

Condition 2 rules out the trivial case where nothing was actually pumped (v and y both empty), and condition 3 is what Section 1.1's proof sketch delivers for free: the inner A was found within the first $m + 1$ levels below the outer one, so vxy – the outer subtree's *entire* yield – is itself bounded by p .

1.2 Reading the Lemma as an Adversary Game

The lemma is a *universal* claim – *every* split of *every* long enough string works – which makes it exactly the wrong shape to use directly. Turn it around: you do not get to pick u, v, x, y, z and call it a proof. You pick one witness string $s \in A$ with $|s| \geq p$; an adversary, standing in for “some parse tree the grammar might have used,” picks the split $s = uvxyz$ afterward, constrained only by $|vy| > 0$ and $|vxy| \leq p$. You win – and A is proved *not* context-free – only if you can defeat *every* split the adversary could possibly choose, by exhibiting, for each one, some $i \geq 0$ with $uv^i xy^i z \notin A$. Losing even one split to the adversary proves nothing; the lemma only promises *some* split works when A really is context-free, so if A is not, no split survives at all.

1.3 Worked Example: $\{a^n b^n c^n : n \geq 0\}$

Suppose $B = \{a^n b^n c^n : n \geq 0\}$ were context-free, with pumping length p . Choose the witness $s = a^p b^p c^p \in B$ (length $3p \geq p$). Let the adversary split $s = uvxyz$ any way it likes, subject to $|vy| > 0$ and $|vxy| \leq p$.

Since $|vxy| \leq p$ and the three blocks a^p, b^p, c^p each already have length p , the substring vxy – a single contiguous window of length at most p – cannot reach from inside the a -block all the way into the c -block; it touches *at most two* of the three letters. So at least one of a, b, c never appears anywhere inside v or y at all. Pump up, $i = 2$: $|vy| > 0$ means $uv^2 xy^2 z$ is strictly longer than s , and every extra character came from v and y – which together miss at least one letter entirely. That letter's count stays fixed at exactly p , while the total length grew, so at least one of the other two letters' counts strictly increased. The three counts can no longer all be equal, so $uv^2 xy^2 z \notin B$. This holds for *every* legal split the adversary could choose, so B is not context-free.

1.4 Worked Example: The Copy Language $\{ww\}$

This is not $\{ww^R\}$ – the reversed-copy language, which *is* context-free, recognized by a PDA that pushes the first half and pops it back off against the second, read in reverse. The plain copy language $D = \{ww : w \in \{0,1\}^*\}$, with no reversal to line up against a stack’s last-in-first-out order, is not.

Suppose D were context-free, with pumping length p . Choose the witness $s = 0^p 1^p 0^p 1^p \in D$ (this is ww for $w = 0^p 1^p$, length $4p \geq p$). Let the adversary split $s = uvxyz$ with $|vy| > 0$, $|vxy| \leq p$. Since $|vxy| \leq p$, and s ’s own midpoint sits at position $2p$, exactly two cases cover every possible split.

Case 1: vxy lies entirely within one half of s . Say it sits inside the second half, $0^p 1^p$ (the first-half case is symmetric). Pump up, $i = 2$: the untouched first half stays exactly $0^p 1^p$, but the whole string is now $|vy| > 0$ characters longer, so the pumped string’s own midpoint – and with it, where its “first half” actually ends – shifts past the original boundary at $2p$, bleeding into the extra characters the pumping just inserted. A first half that no longer stops exactly where the untouched block does can no longer match a second half built from what remains, so $uv^2xy^2z \notin D$.

Case 2: vxy straddles the midpoint, spanning some of the first half’s trailing 1’s and some of the second half’s leading 0’s. Pump down, $i = 0$: this removes characters from exactly that boundary region, leaving $uxz = 0^i 1^j 0^j 1^p$ for some i, j with $i + j < 2p$ (some of the middle 1’s and 0’s are gone, while the untouched 0^p prefix and 1^p suffix survive at full length p each). A string of this exact shape can only equal some $w'w'$ if $i = j = p$ exactly – reconstructing the original, unpumped string – so any other i, j breaks the match, and $uxz \notin D$.

Every split falls into one of these two cases, and both fail the adversary, so D is not context-free.

To think!!

Section 1.3’s proof pumped *up* ($i = 2$); Section 1.4’s Case 2 pumped *down* ($i = 0$) instead, and Case 1 could have used either. Nothing in the lemma says which direction to pump – condition 1 promises *every* $i \geq 0$ works if A is context-free, so defeating the adversary only ever needs *one* value of i that fails. Before starting a new proof, is there a quick way to tell, just from the shape of the language, whether pumping up or pumping down is more likely to break it fastest?

Exercises: Proving Languages Not Context-Free

1. Prove $\{0^n 1^n 0^n 1^n : n \geq 0\}$ is not context-free. (Hint: the witness and the two-case split are close cousins of Section 1.4's – what changes is which blocks stay untouched in each case.)
2. Prove $\{a^i b^j c^k : i < j < k\}$ is not context-free. (Hint: adapt Section 1.3's witness so that all three counts start out strictly increasing, not equal – $a^p b^{p+1} c^{p+2}$, say – and check that pumping in *either* direction breaks the strict inequalities somewhere.)
3. Prove $\{ww^R w : w \in \{0,1\}^*\}$ is not context-free. (Hint: three copies of roughly w instead of two – choose a witness with three easily distinguishable blocks per copy, the way $0^p 1^p$ gave Section 1.4's w two distinguishable blocks, and case-split on which of the three copies vxy falls into.)

2 References

1. (SIPSER) Michael Sipser, *Introduction to the Theory of Computation*, 3rd ed., Cengage Learning, 2012.
2. (HMU) John E. Hopcroft, Rajeev Motwani, Jeffrey D. Ullman, *Introduction to Automata Theory, Languages, and Computation*, 3rd ed., Pearson, 2006.

3 Further Readings

1. (SIPSER) §2.3 “Non-Context-Free Languages,” Theorem 2.34 – the formal statement matching Section 1.1's box exactly, including the Chomsky-Normal-Form proof sketch via a repeated variable on one root-to-leaf path.
2. (SIPSER) §2.3, p.127 – Sipser's own proof that the copy language $D = \{ww : w \in \{0,1\}^*\}$ is not context-free, matching Section 1.4's two-case argument on the same witness string $0^p 1^p 0^p 1^p$.
3. (HMU) §7.2 “The Pumping Lemma for Context-Free Languages” – HMU's version of the same tool and the same adversarial reading of it, applied there to $\{a^n b^n c^n : n \geq 0\}$ among other languages, matching Section 1.3.

4 Disclaimer

The above document is intended to be a reading material/class note for an undergraduate course on Theory of Computation. This first draft may contain errors as it has not been reviewed. Feel free to let me know the gaps, errors, and typos you encounter in the document.